Discussion 5: Morphology For Images

Discussion Topic:

You have seen how to implement morphology for image processing in computer vision. Think of a computer vision application (not already discussed in this module) where morphology is useful in processing the input images to get more reliable results. In a post, address the following:

1. Give an overview of the application and the problem the application seeks to solve.
2. Discuss what morphology operations could be used and how these operations could aid in getting better application results.
3. Discuss what morphology operations could lead to non-optimal results. Why?

My Post

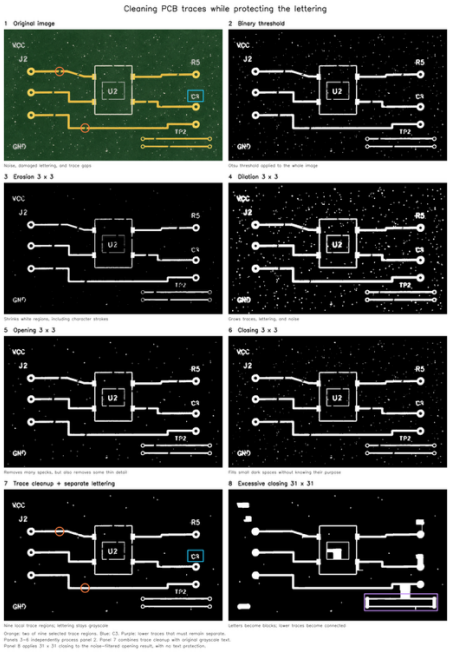
Hello Class,

1 - Overview of the application and the problem

I build small printed circuit boards (PCBs) as a hobby, so I decided to use a PCB image for this discussion. Sometimes I find a board online that I would like to reproduce, but the picture is difficult to read. I can follow most of the traces, which are the copper paths between connections, and then come across a section that looks disconnected. The small letters and numbers can be a problem too. A marking such as C3 is not very useful if I can see the C but cannot tell what comes after it.

For this example, I used a PCB image, a PCB drawing image, with bright specks, worn text, and missing sections in some traces; see Figure 1. The application here is image preprocessing to help inspect a board layout.

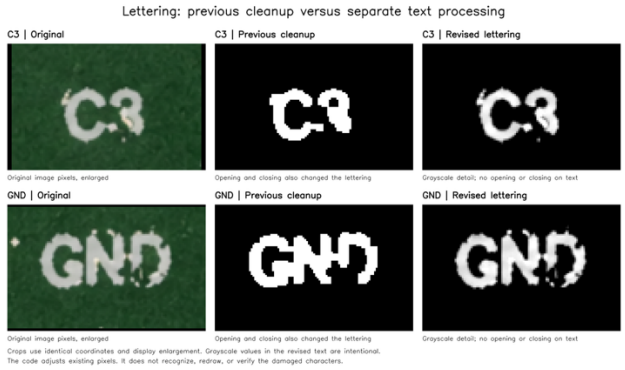
Figure 1
Morphological Processing of an Illustrative Degraded PCB Image



Note. Orange circles identify two selected image gaps; the blue box identifies the C3 marking; and the purple box identifies the two lower traces. Panels 3–6 independently process the binary image in panel 2. They are not successive steps.

Figure 2

Lettering Before and After Separate Text Processing



Note: Each row shows the same text region from the original image, the first cleanup, and the final result. The final result uses the original grayscale pixels rather than the lettering already changed by opening and closing.

2 - Morphological operations

I resized the image to 1200 × 720 pixels, converted it to grayscale, and used Otsu thresholding to produce white traces and lettering against a black background. Note that morphology can be defined as the process of working with shapes in an image using a structuring element, or kernel (Colorado State University Global, n.d.).

To morphologically process the image, I first used a 3 × 3 square kernel. The Erosion operation removed many small specks, but it also made the lettering thinner. The Dilation operation went in the other direction: the traces became thicker, dilated, and so did the unwanted white spots. These morphological operations act on the foreground pixels; they do not distinguish a letter shape from noise (OpenCV, n.d.). Other morphological operations are a combination of these two basic operations, erosion and dilation. For example, opening combines erosion followed by dilation, while closing applies them in reverse order (The scikit-image contributors, n.d.). Opening and closing, or Erosion and Dilation, are not inverse operations, and applying Closing after Opening does not necessarily reproduce the original image because some of the pixels removed during Opening cannot be recovered; they are lost.

In my example, I followed opening with removal of isolated white regions smaller than 16 pixels and then 3 × 3 closing. For the trace repairs, I used a horizontal 13 × 3 kernel in nine manually selected regions, including the two gaps marked in Figure 1. I could have applied the same kernel to the entire image, although that would also affect the other traces and the small letters. In this case, I only wanted to close gaps along those traces, without connecting traces that should remain separate. I processed the lettering separately from the original grayscale image. The following code shows how I applied these operations; see Code Snippet 1.

```
# Apply horizontal closing in the nine manually selected trace regions.
# TRACE_GAPS stores their center coordinates in the complete script.
trace_kernel = cv2.getStructuringElement(cv2.MORPH_RECT, (13, 3))
for x, y in TRACE_GAPS:
    x0, y0, x1, y1 = x - 22, y - 15, x + 22, y + 16

    # NumPy uses [rows, columns], corresponding to [y, x].
    roi = trace_cleaned[y0:y1, x0:x1]
    trace_cleaned[y0:y1, x0:x1] = cv2.morphologyEx(
        roi, cv2.MORPH_CLOSE, trace_kernel
    )

# Use the ORIGINAL grayscale text, not the trace-processed lettering.
cleaned = trace_cleaned.copy()
for x0, y0, x1, y1 in TEXT_REGIONS:
    text_roi = gray[y0:y1, x0:x1]

    # Adjust contrast and remove tiny isolated regions.
    # clean_text() does not erode, dilate, open, or close the letters.
    cleaned[y0:y1, x0:x1] = clean_text(text_roi)

# Compare with excessive closing without separate text processing.
k31 = cv2.getStructuringElement(cv2.MORPH_RECT, (31, 31))
too_much = cv2.morphologyEx(filtered, cv2.MORPH_CLOSE, k31)
```

Note. Code Snippet 1 shows the part of the program I used to repair the selected trace sections and process the lettering separately. The complete script also loads the image, converts it to grayscale, applies Otsu thresholding, and generates Figures 1 and 2. I selected the coordinates in TRACE_GAPS and TEXT_REGIONS manually; the program does not identify the trace gaps or recognize the letters. For the lettering, the clean_text() function uses the original grayscale values, adjusts their contrast, and removes isolated regions smaller than three pixels. Unlike the morphological operations applied to the traces, this method keeps the intermediate gray intensity values of the remaining letter pixels.

3 - Morphological non-optimal results

Looking at Figure 1, I think opening did a fairly good job of removing the noise from the background. In the original picture, some of the white spots are very close to the traces, making it more difficult to distinguish the actual circuit paths from the noise. After processing, most of those spots are gone, and I can follow the traces more easily. Additionally, the two missing sections marked with orange circles appear connected in the processed image. Note that I selected these locations manually, based on the original picture. The program itself does not determine whether those sections are supposed to be connected.

The lettering was a bit more difficult. Initially, I thought that closing would also help with the missing pixels in the letters, since the operation can fill small gaps. However, looking at Figure 2, I found that the first result was actually worse than the original in some areas. For example, parts of GND became joined together, and the number 3 in C3 lost some of its original shape. Instead of trying to correct those letters after the morphological operations had already modified them, I decided to use the original grayscale image and process the lettering separately. I adjusted the contrast and removed very small isolated regions, without applying opening or closing to the letters. Note that contrast adjustment is not a morphological operation. I used this alternative because the morphological processing was removing or joining pixels that were important for recognizing the characters.

In Figure 2, the revised lettering retains more of its original shape, although the number in C3 is still somewhat difficult to recognize. The contrast adjustment makes the remaining strokes more visible, but it cannot reproduce the parts of the character that are missing from the original image. There is also another problem with removing small isolated regions. If a few pixels belonging to a letter have become separated from the rest of the character, the program may remove those pixels as noise. This means that even when the processed image looks cleaner, it is possible that some useful information has been removed. For this reason, I would still compare the results with the original image, especially when trying to identify a damaged component marking.

I also experimented with a larger closing kernel of 31 × 31, applying it to the noise-filtered binary image. I wanted to see whether increasing the kernel size would help close more of the missing sections in the traces. Looking at Figure 1, the result shows why that would not be a good idea. The two independent traces near the bottom of the PCB become connected, and some of the lettering loses its shape almost completely. Closing applies dilation followed by erosion, which can fill small dark regions between foreground pixels. The problem is that the operation does not know whether a dark region represents missing image information or an intentional separation between two objects (OpenCV, n.d.; The scikit-image contributors, n.d.). Erosion can also produce undesirable results, but in a different way. Since it removes pixels from the boundaries of foreground regions, thin traces or parts of letters may disappear, as illustrated in the module lecture with the text example (Colorado State University Global, n.d.).

What I find interesting about this example is that morphology can make the PCB image easier to examine while also changing information that is important for understanding the circuit. For example, two copper traces may appear very close to each other in a picture, but that does not necessarily mean they should be connected. If closing joins those traces, the processed image would no longer represent the same circuit layout. This would be a problem if I were using the picture to reproduce the PCB, since I could end up making an electrical connection that was never supposed to exist. Nonetheless, I would still use morphology to remove noise and make the traces easier to follow, while using the separate grayscale processing for the lettering. I would also keep the original image for comparison. If I could not determine whether two traces were connected, I would try to find the circuit schematic or another

picture of the board before reproducing that connection. The processed image would help me examine the circuit, but I would not use it alone to determine how the components should be connected

-Alex

References

Colorado State University Global. (n.d.). *Lecture 5: Morphology* [Course lecture]. Canvas.

OpenCV. (n.d.). *Morphological transformations*. https://docs.opencv.org/4.13.0/d9/d61/tutorial_py_morphological_ops.html

The scikit-image contributors. (n.d.). *Morphological operations*. https://scikit-image.org/skimage-tutorials/lectures/3_morphological_operations.html